ICT Unit D: Basic Programming Concepts

Ch21 Problem-solving Procedure

➢ Problem-solving concepts

- **Problem-solving**: Process of analyzing the description of a problem until we reach a solution
- **Divide and Conquer** (A strategy for solving a problem): Problem is broken into some smaller modules
  → Solve smaller problems (Sub-problem) individually (Include: Top-down approach, Bottom-up approach)
- **Stepwise refinement**: The process of breaking down a programming problem into a series of steps
- **Solving a problem in an organized + systematic way** → Greatly increase the chance of finding the best solution

➢ Problem-solving procedures

Basic procedures

1. Problem identification	⇒ Obtain a **precise definition** of a problem
2. Problems analysis (In-depth considerations of the defined problem)	IPO Chart: **Inputs + Processing steps + Outputs** required
3. Algorithm design **4. (logic & action)**	• **Algorithm**: Finite sequence of steps arranged in specific logical order → Produce a solution • **Pseudo-code** (Narrative form) / **Flowcharts** (Graphical form) → Design + Represent algorithm
5. Develop a solution	• **Top-down approach**: Initial problem is divided into some sub-problems • **Structure diagram**: Modules (With module specification)
6. Debugging and testing	• **Debugging**: Locating and fixing defects in programme • **Testing**: Ensure that a program works + free of error • **Error**: Syntax + Run-time + Logic
7. Documentation (Created throughout the programming process)	• Describe in **detail purpose** of a program + [How it's Designed + Developed + Tested] - ✓ **Maintenance** + ✓ **Familiarize** - ✓ Discover **errors** + ✓ Learn the program (End-user) • Types: **User manuals + Program manuals** (Test data, Pseudo-code, Flowcharts)

➢ Different ways to solve the same program

- Solve systematically: Top-down approach (Large programme) + Bottom-up approach (Small programme)

➢ Extra

- Module specification → Helps understand how data is transmitted between modules
  Input (Data) → Process steps (Logic) → Output (Information)

Carousell@ose_note_2025xx

Ch22 + Ch23 Algorithm Design

➢ Introduction to algorithm design

- **Design an algorithm**: Design how a problem can be solved
- **Pseudo-code + Flowcharts**: Independent of programming language

➢ Identifying input and outputs of a problem

- IPO Charts (IPO of a program)

  ⇒ **Define the requirement for IPO**: Types + Precision levels + Units + .etc(Constraints)

- User-interface(UI) (User friendliness)

1. Form field validation	Fulfilling the **input requirement**
2. Familiar placement of elements on UI	e.g. English text should be from left to right
3. Well-designed layouts	• **Colour**: Appropriate • **Control objects** (Radio button, Check-boxes): Appropriate • **Alignment**: Consistent for fields • **Wording**: Concise + appropriate
4. UI design consistency	↓ **Cost** of training + technical support (∵ More familiarize with how the programme works)
5. Fixed path of navigation	↓ **Chance** of getting lost in in the UI
6. Different versions	↑ **Efficiency** of the use of UI (∵ Tailor-made 定制 for different types of users)

➢ Basics of constants and variables in computer programming

- **Constant / Variable** (Δ /Δ = 0)) | **Data** = Unorganized facts | **Simple data type**: Integer, char, boolean, real number
- **String**: Collection of char
- **Array**: Collection of variables of the same data type + can be accessed by the same number
- **Boolean** can be manipulated using [relational operator (>, <) + logical operator (AND, OR)]
- Data structure (Storing method of data) → Data can be found + used efficiently
  e.g String, 1-D Array
- **Truth table** → List all combinations of [input + resultant] values with logical operator
- Algorithm makes decision ← Evaluate the value of logical expression

➢ Selection of appropriate data types

- **Data type**: Numeric (Integer / Real) + Character (Char, String) + Logical (Boolean)
- **Logical data**: For making 'Yes' , 'No' or 'True' , 'False'

p	q	p ∧ q
T	T	T
T	F	F
F	T	F
F	F	F

➢ Different statements

1. **Input statement**: Obtain input + Store data (obtained in a variable)
2. **Output statement**: Output processed input
3. **Assignment statement**: Place a value / value of an expression → variable

➢ Basic control structures

- **Sequence control structure** (Steps in sequence)
- **Selection control structure** (Steps in selection)
- Conditional expression = Logical expression

Carousell@dse_note_2025xx

1. Decision logic structure	Use IF_THEN_ELSE statement + Conditional expression
2. Case logic structure **(n option → n IF statement)**	Make 1 decision from n possible values of an expression
3. Iteration **(Repeat same block of expression → Condition is T/F)**	• **Definite loop** - FOR_LOOP: Increment 自增 loop / Decrement loop • **Infinite loop** - Post-test loop: DO_WHILE_LOOP/ REPEAT_UNTIL_LOOP - Pre-test loop: WHILE_DO_LOOP

➢ Algorithm for searching arrays

- **Initialization**: values of a variable / constant
- **Searching a particular element in an array** (Check existence ; Locate position → Deletion + Insertion)
  1. **Linear search**: Examine all elements of an array until it's found (From 1st one → For unsorted list)
  2. **Binary search**: Examine the middle element (From $[(n+1) \text{ div } 2]^{th}$ one → For sorted list)

➢ Advantages of modular approach

1. **Re-usability** for frequently used module (Procedure)
2. Easier → **Understand+ Debug** (Small module < Single whole programme)
3. Easier → **Monitor + Control** (Large project)
4. **Boost** Development process

Ch24 Algorithm Testing

➢ Tracing & Testing algorithms

- Ensure the programme fulfils its purpose

1. **Dry run** (on paper)
   - Simulate the execution of the program on paper
     ⇒ Examine the **Logic + Code** of a programme

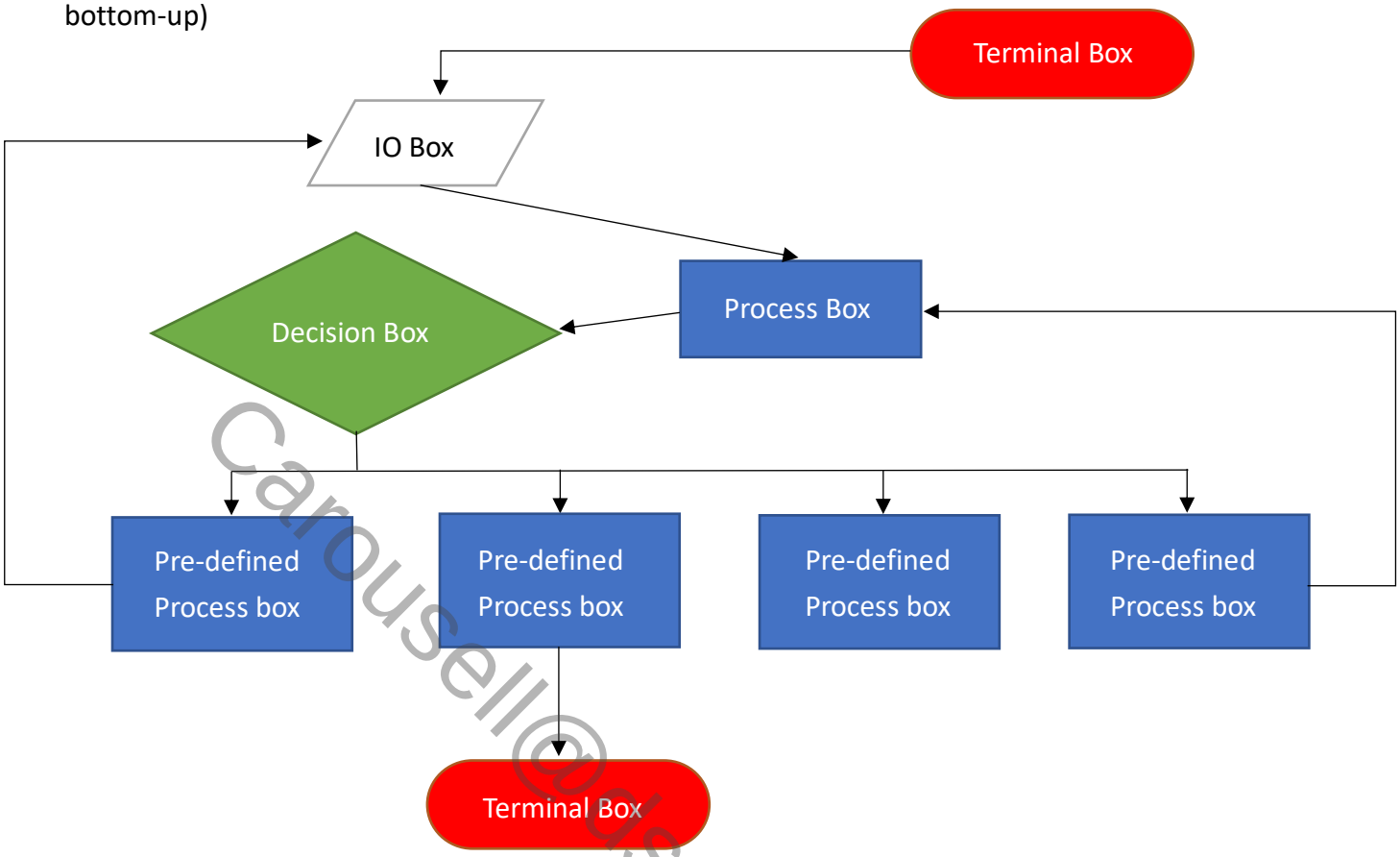
   Steps
   1. Prepare a set of **text data**
   2. Create a **tract table**
   3. **Input** data (Text data)
   4. **Execute** on paper (1 line once)
   5. **Record** the values on trace table

   Trace table → List the values of the variables through running 1line/time of the programme
2. **Test data** (on paper or anythings)
   - Generated intentionally
     ⇒ Check the **accuracy** of a programme (Correct **Program design** + Able to handle **improper data**)
     - **Types of test data**
       i. Whole range of **valid data**
       ii. Critical values (for **boundary case**)
       iii. Invalid data
3. **Echo checking** (on computer, manual debugging tool)
   - **Scaffolding**(使新手能夠執行他們無法完成的任務): Output contents of variable and / or parameters 參數

Carousell@dse_note_2025xx

➢ Comparing different solutions to the same problem

- **In different aspects**: Resource usage + Steps in operation + Problem-solving approach (Top-down, bottom-up)



Carousel@dse_note_2025xx